Auction House Project

1. Project Overview

Build a production-grade online auction platform supporting multiple auction formats including English, Dutch, and Vickrey auctions. Users can create listings, participate in live bidding, and receive real-time updates through WebSockets. The system emphasizes correctness, scalability, and production-ready engineering with CI/CD, monitoring, automated testing, and AI-powered search and fraud detection. It is designed to demonstrate backend engineering, distributed systems, and applied AI skills.

2. Scope & Features

- JWT authentication, role-based access control, user profiles
- Auction creation, editing, categories, image uploads
- English, Dutch, and Vickrey auction types
- Real-time bidding with WebSockets
- Automatic winner selection and auction closing
- Notifications for outbid, win/loss, and auction end
- Search, filters, sorting, pagination
- Natural-language search over listings
- Semantic search using embeddings/vector database
- AI spam/fraud listing detection
- Redis caching and rate limiting
- PostgreSQL, SQLAlchemy, Alembic migrations
- Background jobs and schedulers
- Audit logs and transaction safety
- Comprehensive unit/integration/API tests
- Docker, GitHub Actions CI/CD, deployment
- Monitoring with Prometheus and Grafana, centralized logging

3. Learning Outcomes

- Design scalable backend architectures with FastAPI.
- Model complex relational databases and transactions.
- Implement concurrent real-time systems using WebSockets.
- Handle race conditions, optimistic locking, and consistency.
- Write production-quality tests and CI/CD pipelines.
- Deploy containerized applications with Docker.
- Use Redis, background workers, and monitoring.
- Apply NLP embeddings and vector search.

• Integrate AI-based fraud detection into a backend system.
• Practice software engineering, Git workflows, and clean architecture.

4. Six-Week Detailed Roadmap

Week 1

Day1: Requirements & architecture. Day2: FastAPI setup, project structure. Day3: PostgreSQL + SQLAlchemy. Day4: Alembic migrations. Day5: JWT auth. Day6: User/profile APIs. Day7: Refactor & tests. Checklist: auth, DB, migrations, project skeleton.

Week 2

Day1: Auction schema. Day2: CRUD APIs. Day3: Image upload. Day4: English auction bidding. Day5: Validation. Day6: Scheduler to close auctions. Day7: Testing. Checklist: working English Auctions.

Week 3

Day1: WebSockets. Day2: Live bidding. Day3: Notifications. Day4: Redis cache. Day5: Search/filter/pagination. Day6: Rate limiting. Day7: Integration tests. Checklist: real-time platform.

Week 4

Day1: Dutch auction engine. Day2: Price-drop scheduler. Day3: Vickrey auction logic. Day4: Audit logs. Day5: Transactions & concurrency. Day6: Optimistic locking. Day7: Testing. Checklist: all auction types.

Week 5

Day1: Embeddings. Day2: Semantic search. Day3: Natural-language search API. Day4: Fraud/spam classifier. Day5: Admin dashboard APIs. Day6: API documentation. Day7: Testing. Checklist: AI features complete.

Week 6

Day1: Docker. Day2: GitHub Actions CI/CD. Day3: Prometheus. Day4: Grafana dashboards. Day5: Performance optimization. Day6: Final bug fixes. Day7: Deployment, README, demo video.
Checklist: production-ready release.